

SOLID Mapping - Detailed Design Justification

Scope: Inventory/Reservation, Payment, Order and Checkout modules of SALESTORM. **How to read this document:** for each principle we give (1) the rule in one line, (2) the SALESTORM classes it governs, (3) a "bad design" that would have caused a real flash-sale failure, (4) the corrected design with code, (5) how a test proves it, and (6) the trade-off we accept.

1. Module Overview (who depends on whom)

Module	Core classes	Depends on (abstractions only)	Must NOT know about
Inventory / Reservation	ReservationService, Reservation, ReservationState	InventoryRepository, ReservationRepository, StockGate, EventPublisher, PurchaseLimitPolicy, Clock	Payments, orders, HTTP, Kafka, Redis client types
Payment	PaymentService, Payment, PaymentStateMachine	PaymentRepository, PaymentProviderFactory, PaymentProvider, EventPublisher, IdGenerator	Inventory tables, orders, which gateway is in use
Order	OrderService, Order, OrderState	OrderRepository, OrderEventObserver, EventPublisher	Gateway APIs, stock counters, email/SMS
Checkout	CheckoutFacade, CheckoutSaga, PricingEngine	InventoryClient, PaymentClient, PricingStrategy, SagaRepository	SQL, HTTP clients, concrete providers

Dependency direction: **Controllers** → **Facade/Saga** → **Services** → **Interfaces** ← **Implementations (infrastructure)**. Domain code never imports infrastructure packages.

2. S - Single Responsibility Principle

Rule: a class has exactly one reason to change.

2.1 Where applied

Class	Single responsibility	Reason to change (only)	What it deliberately does NOT do
ReservationService	Stock lifecycle: reserve, confirm, release, expire	Reservation business rules change	Charge cards, create orders, send emails
PaymentService	Payment state and gateway interaction	Payment rules change	Know about orders or stock
OrderService	Order lifecycle and history	Order rules change	Charge, reserve, notify
NotificationObserver	Compose and send customer messages	Message channel/template changes	Change order state
PricingEngine	Compute payable amount	Pricing/promotion rules change	Reserve stock
IdempotencyGuard	Detect and replay duplicate requests	Dedupe policy changes	Execute business logic
ExpirySweeper	Find expired reservations and invoke release	Expiry schedule/batch size changes	Decide release rules (calls ReservationService.expireDue)
OutboxPublisher	Persist events with state change	Event format changes	Decide which events exist

2.2 Bad design (what we avoided)

```
class CheckoutManager {
    // 6 reasons to change
    void buy(Request r) {
        db.execute("UPDATE inventory ..."); // stock
        gateway.charge(r.card, amount); // payment
        db.execute("INSERT INTO orders ...");// order
        smtp.send(r.email, "Thanks!"); // notification
        courier.book(r.address); // shipping
    }
}
```

Failure consequences in a flash sale: a courier API outage rolls back or blocks the reservation; changing the email template risks oversell; the whole class is untestable without a database, a gateway and an SMTP server; a crash between charge and order creates a paid-but-no-order customer with no recovery path.

2.3 Corrected design

```
class ReservationService { Result reserve(ReserveCommand c) { ... } } // stock only
class PaymentService { PaymentResult initiate(PaymentCommand c) { ... } } // payment only
class OrderService { void createFromPayment(PaymentSucceeded e) { ... } } // order only
class NotificationObserver implements OrderEventObserver { void onEvent(OrderEvent e) { ... } }
```

Services communicate through **events** (PaymentSucceeded, OrderConfirmed) so each can fail, retry and scale independently.

2.4 Test evidence

- ReservationServiceTest runs with in-memory fakes and **no** payment/order classes on the classpath (compile-time proof of separation).
- Failure injection: kill the notification consumer → orders still reach CONFIRMED.

2.5 Trade-off

More classes and more messages; a developer must follow an event chain instead of a single method. Mitigated by distributed tracing and an event catalogue (05_API/02_Event_Design.md).

3. O - Open/Closed Principle

Rule: open for extension, closed for modification.

3.1 Extension points in SALESTORM

Extension point (abstraction)	Existing implementations	Example future extension	Core code changed?
PaymentProvider	RazorpayAdapter , StripeAdapter	PhonePeAdapter , UpiDirectAdapter	No (register in factory config)
PricingStrategy	FlashSalePricing , CouponPricing	BulkDiscountPricing , LoyaltyPricing	No
DeliveryPartner	DelhiveryAdapter , BlueDartAdapter	ShiprocketAdapter	No
OrderEventObserver	NotificationObserver , ShipmentObserver	AnalyticsObserver , WhatsAppObserver	No
PurchaseLimitPolicy	OnePerCustomerPolicy	VipEarlyAccessPolicy	No
StockGate	RedisStockGate	InMemoryStockGate (tests), BucketedStockGate	No

3.2 Bad design

```
if (provider.equals("razorpay")) { ... }
else if (provider.equals("stripe")) { ... }      // edit PaymentService for every new provider
```

Risk: editing the payment core just before a sale to add a provider can break the working path.

3.3 Corrected design - adding a provider step by step

1. Create PhonePeAdapter implements PaymentProvider (translate request/response, map error codes, always send merchantRef).
2. Register: factory.register("PHONEPE", PhonePeAdapter::new) or a config entry payment.providers.phonepe.enabled=true .
3. Run the **provider contract test suite** (same suite every adapter must pass; see LSP).
4. Done. PaymentService , CheckoutFacade , PaymentRepository , database schema: **unchanged**.

3.4 Trade-off

Abstractions add indirection and can hide provider-specific features (e.g. UPI intent flows). We allow an optional capabilities() method so callers can check support without instanceof .

4. L - Liskov Substitution Principle

Rule: any implementation must be usable wherever its abstraction is expected, without callers knowing.

4.1 PaymentProvider contract (every implementation must honour)

Rule	Detail
Preconditions	amount > 0 , non-empty merchantRef ; implementations may not add stricter preconditions
Idempotency	Calling charge twice with the same merchantRef never creates two charges
Return values	charge returns PENDING , SUCCESS or DECLINED ; business declines are return values , not exceptions
Exceptions	Only TransientGatewayException (retryable) and PermanentGatewayException ; never UnsupportedOperationException
Timeouts	Honour the caller's deadline; never block indefinitely
query(merchantRef)	Always available (needed for reconciliation); returns UNKNOWN if the gateway has no record
refund	Supported by every provider (a provider that cannot refund is not eligible to register)

4.2 Substitutable implementations

RazorpayAdapter , StripeAdapter , ResilientProviderDecorator(wrapping any provider) , FakeProvider (tests), ChaosProvider (injects latency/failures in load tests). PaymentService behaves identically with all of them.

4.3 Violation example we prevent

```
class LegacyBankAdapter implements PaymentProvider {
    RefundResponse refund(RefundRequest r) { throw new UnsupportedOperationException(); } // breaks reconciliation & compensation
}
```

This would make the saga's compensation path crash in production. Rule: if a provider cannot refund, split the interface (ISP) and do not register it for flows that need refunds.

4.4 Test evidence - shared contract test

```

abstract class PaymentProviderContractTest {
    abstract PaymentProvider provider();
    @Test void sameMerchantRefIsIdempotent() { /* charge twice, expect one transaction */ }
    @Test void declineIsReturnedNotThrown() { /* card 4000..0002 -> DECLINED */ }
    @Test void queryUnknownRefReturnsUnknown() { }
    @Test void honoursDeadline() { }
}
class RazorpayAdapterTest extends PaymentProviderContractTest { ... }
class StripeAdapterTest extends PaymentProviderContractTest { ... }

```

Also applied to `StockGate` (`RedisStockGate`, `InMemoryStockGate`): `tryAcquire` is atomic, never goes below zero, `giveBack` is idempotent per token.

4.5 Trade-off

Strict contracts restrict what an adapter may do and require maintaining a shared test suite; the benefit is that provider swaps during an incident are safe.

5. I - Interface Segregation Principle

Rule: clients should not depend on methods they do not use.

5.1 Where applied

Fat interface avoided	Focused interfaces used	Client that benefits
<code>InventoryService</code> (reserve, release, sold, report, admin, import...)	<code>StockGate</code> (2 methods), <code>InventoryRepository</code> (3), <code>ReservationRepository</code> (3), <code>ExpiryRepository</code> (1)	<code>ExpirySweeper</code> needs only <code>findExpired + release</code>
<code>PaymentGateway</code> (charge, refund, query, settle, webhook)	<code>Chargeable</code> , <code>Refundable</code> , <code>Queryable</code> , <code>WebhookVerifier</code>	<code>PaymentReconciler</code> depends only on <code>Queryable</code>
<code>OrderManager</code>	<code>OrderCommands</code> (create/advance/cancel), <code>OrderQueries</code> (get/list)	Customer UI uses queries only; CQRS-friendly
<code>NotificationService</code>	<code>OrderEventObserver</code> (1 method)	Each observer implements one method

5.2 Code

```

interface StockGate { boolean tryAcquire(ProductId p, int qty); void giveBack(ProductId p, int qty); }
interface Queryable { ChargeStatus query(String merchantRef); }
interface Refundable { RefundResponse refund(RefundRequest r); }
interface Chargeable { ChargeResponse charge(ChargeRequest r); }
class PaymentReconciler { PaymentReconciler(Queryable q, PaymentRepository repo) { ... } } // cannot charge by mistake

```

5.3 Test evidence

`PaymentReconciler` unit tests mock a one-method interface (`Queryable`); no unused stubs required. A compile-time guarantee that a reconciliation job can never trigger a new charge.

5.4 Trade-off

More small interfaces to name and wire; mitigated by implementing several on one class (`RazorpayAdapter` implements `Chargeable`, `Refundable`, `Queryable`).

6. D - Dependency Inversion Principle

Rule: high-level policy depends on abstractions; low-level details depend on the same abstractions.

6.1 Dependency map

```

CheckoutSaga → InventoryClient (interface) ← HttpInventoryClient / InProcessInventoryClient
ReservationService → InventoryRepository ← PostgresInventoryRepository
ReservationService → StockGate ← RedisStockGate
ReservationService → EventPublisher ← KafkaOutboxPublisher
ReservationService → Clock ← SystemClock / FixedClock (tests)
PaymentService → PaymentProvider ← adapters

```

6.2 Composition root (only place that knows concrete classes)

```

var gate = new RedisStockGate(redisClient);
var invRepo = new PostgresInventoryRepository(dataSource);
var resRepo = new PostgresReservationRepository(dataSource);
var events = new KafkaOutboxPublisher(outboxRepo);
var service = new ReservationService(invRepo, resRepo, gate, new OnePerCustomerPolicy(), events, Clock.systemUTC());

```

6.3 Why it matters for a flash sale

Situation	Benefit of DIP
Redis outage	Swap <code>RedisStockGate</code> for a <code>PassThroughGate</code> by configuration; domain code unchanged
Time-based tests (10-minute expiry)	Inject <code>FixedClock</code> ; test expiry without sleeping

Concurrency simulation	Inject in-memory repositories to test logic quickly
Gateway change	Only the factory registration changes

6.4 Test evidence

Unit tests for `ReservationService` run in milliseconds with fakes; an architecture test (ArchUnit) enforces "domain packages must not import `redis.*`, `org.postgresql.*`, `kafka.*`".

6.5 Trade-off

Wiring complexity and indirection; accepted because testability and swap-ability are central to the reliability requirements.

7. Summary Matrix

Principle	Primary classes	Failure it prevents	Verification
SRP	Reservation/Payment/Order services, observers	One change breaking unrelated flows; untestable god class	Separate unit tests, no cross-module imports
OCP	Provider, pricing, delivery, observer, policy interfaces	Editing core code to add a provider	New adapter only; contract tests
LSP	<code>PaymentProvider</code> , <code>StockGate</code> implementations	Compensation crash from <code>UnsupportedOperationException</code>	Shared contract test suite
ISP	<code>Chargeable</code> , <code>Queryable</code> , <code>Refundable</code> , <code>StockGate</code>	Reconciler able to charge; heavy mocks	Compile-time dependencies
DIP	All services via interfaces; composition root	Hard-wired Redis/DB/Kafka; slow tests	ArchUnit rule + in-memory fakes

8. Extension Scenarios (minimal change)

New requirement	Steps	Classes added	Classes modified
Add PhonePe	Adapter + factory registration + contract test	<code>PhonePeAdapter</code>	none
Add "Buy 2 get 10%"	New strategy + config	<code>BulkDiscountPricing</code>	none
New courier	Adapter implementing <code>DeliveryPartner</code>	<code>ShiprocketAdapter</code>	none
WhatsApp notification	New observer subscribed to <code>OrderConfirmed</code>	<code>WhatsAppObserver</code>	none
VIP early access	New policy	<code>VipEarlyAccessPolicy</code>	none
Stock bucketing for large SKUs	New gate	<code>BucketedStockGate</code>	none

9. Jury Q&A

Q: Isn't this over-engineered for one product? The flash sale is the hardest case; the interfaces cost little and each one maps to a real failure or a real extension (provider outage, tests, new payment method). **Q: Which principle matters most here?** DIP and LSP: both make failure handling safe (swap Redis out, swap provider during an outage, mock time). **Q: Show a violation you avoided.** A fat `PaymentGateway` interface would let the reconciliation job call `charge`, risking double charge; ISP prevents it.